Inheritance, Polymorphism, Interfaces and Exceptions

Object-Oriented Programming - Units 2-3 - Reuse and robustness

Inheritance in practice

```
abstract class Shape {
    abstract double area();
    public String toString() { return getClass().getSimpleName()
        + " area=" + area(); }
}
class Circle extends Shape {
    double r; Circle(double r) { this.r = r; }
    double area() { return Math.PI * r * r; }
}
```

- Single, multilevel and hierarchical inheritance are supported by most languages; Java avoids multiple class inheritance but allows multiple interfaces.
- super refers to the parent class; use it to call a parent constructor or overridden method.

Overloading vs overriding

```
Overloading                    Overriding
Same class, same name          Subclass redefines parent method
Different parameter lists      Same signature and return type
Resolved at compile time       Resolved at run time (dynamic dispatch)
```

Interfaces vs abstract classes

- An interface declares a contract; a class may implement many interfaces.
- An abstract class can hold state and partial implementation but is single-inherited.
- Prefer composition over inheritance when the relationship is has-a rather than is-a.

Exception handling

Exceptions separate error handling from normal logic. Use try to guard risky code, catch to handle specific failures, finally for cleanup that must always run, and throw to signal an error. Checked exceptions must be declared or handled; unchecked ones (RuntimeException) indicate programming errors.

```
try (var in = new Scanner(new File("data.txt"))) {
    System.out.println(in.nextLine());
} catch (FileNotFoundException e) {
    System.err.println("Missing file: " + e.getMessage());
}
```

SOLID in one line each

- Single responsibility, Open/closed, Liskov substitution, Interface segregation, Dependency inversion.